Freight Rate Prediction

Spotter - Machine Learning Engineer assessment | 26 October 2026

Summary

The task is pure forecasting, not interpolation: training covers 1 January to 31 October 2025 and the 12,000 loads to be predicted cover 1 November to 31 December, with no date overlap. Every design decision below follows from that, starting with a validation scheme that reproduces the forward gap rather than a random split that hides it.

The submitted model is **ridge regression on log(posted_rate)**. It was selected against a LightGBM alternative on three expanding-window folds and won every metric (MAE $119 against $131, MAPE 5.16% against 5.87%). That is the opposite of the expected result and the reason is given in section 6; the gradient-boosted model is kept in this report as the documented comparison rather than quietly dropped.

1. What the data looks like

- **No overlap in time.** 48,000 labelled rows to 31 October; 12,000 unlabelled rows after it (5,836 November, 6,164 December).
- **Distance dominates price.** Raw *posted_rate* runs $57.22 to $25,533 with a coefficient of variation of 0.626, but rate per mile sits in a tight band: 1.67 / 2.15 / 3.18 $/mi at p1 / p50 / p99. Most apparent outliers are long hauls, not errors.
- **A short-haul premium.** Mean rate per mile falls from $2.815 on sub-250-mile loads to $1.903 on 2,500-3,500-mile loads.
- **Gentle seasonality.** Monthly rate per mile climbs 2.10 in January to a June peak of 2.33 and eases to 2.24 by October. Day of week is smaller: 2.24 midweek against 2.19 at the weekend. Real signal, no regime breaks.
- **Coordinates are redundant.** Every city maps to exactly one latitude/longitude pair, so the four coordinate columns are a deterministic lookup of the city name.
- **Lanes are thin.** 4,014 observed lanes over 48,000 rows, median 10 loads each - far too few for a per-lane target encoding, which would overfit.
- **The two market signals are not duplicates.** *market_index* and *quote_signal* correlate -0.07 with each other, so both are kept.

2. Data quality, and how each issue was handled

Issue	Scale	Decision and evidence
Negative weight	292 train 145 validation	Take the absolute value. Their absolute values average 31,724 lb against 31,415 lb for positive rows - the same distribution, flipped. Sign errors, not a separate population.
Missing weight	300 train 165 validation	Impute with the training mean and add an indicator column. Dropping is impossible: all 12,000 load IDs must be returned.
Missing market_index	374 train 249 validation	Same treatment. Every statistic is fitted on training data only, then applied unchanged to validation and December.
Extreme rate per mile	96 train rows	Trim outside the 0.1st-99.9th percentile of rate per mile, inside the training fold only. Trimming a validation fold would delete the hard rows and flatter every metric.
Unseen cities	1,447 validation rows	8 of 72 cities never appear in training. They are flagged for the model rather than silently encoded as a new level.

3. The train/test split

Training ends 31 October and prediction begins 1 November, so the validation scheme has to reproduce that forward gap. Three expanding-window folds, each training on everything before its validation month and scoring the month immediately after:

Fold	Train on	Train rows	Validate on	Rows
1	January - July	33,718	August	4,759
2	January - August	38,477	September	4,670
3	January - September	43,147	October	4,853
final	**January - October**	**48,000**	**November - December**	**12,000**

Fold 3 validates on October, the month closest in shape to the real task, so it is the fold used to early-stop the boosted model. Within each fold the round count is chosen on an inner holdout - the last month of that fold's own training window - never on the month being scored.

Why not a random KFold

A shuffled split puts October rows into training while scoring August, so the model sees the future it is being asked to predict. Measured on this data it flatters LightGBM by 4.9% on RMSE and 1.6% on MAPE. The gap is modest because the time signal in this data is gentle, but it is optimism bought for nothing, and a random fold also has no last month to early-stop against.

4. Target transform

Fitting squared error on raw dollars lets a handful of 3,000-mile hauls dominate the loss. Two candidates were measured:

Target	CV	Skew	Excess kurtosis
posted_rate (raw)	0.626	+1.902	+13.28
log(posted_rate)	**0.088**	**-0.491**	**+0.14**
posted_rate / distance	0.264	+8.720	+113.69
log(posted_rate / distance)	0.250	-0.039	+29.45

log(posted_rate) was chosen, for two reasons. It is far better conditioned - near-normal, against a rate-per-mile target whose excess kurtosis is +114 because short hauls divide a fixed cost by a small distance. And it does not assume the answer: regressing log(posted_rate) on log(distance) gives a slope of **0.871**, not 1.0, which is the short-haul premium. A rate-per-mile target hard-codes that exponent at exactly 1.0 and leaves the model to undo the error. Predictions are inverted with exp and a Duan smearing factor fitted on training residuals, without which exp of a mean of logs returns the geometric mean and every prediction comes back low.

5. Features

24 features, all computable for a December row that lacks coordinates and both market signals:

- **Size and scale:** distance, log(distance), weight, weight per mile.
- **Circuity:** distance divided by the great-circle distance between the two cities. The raw haversine figure is useless alone (r = 0.9995 against the supplied distance) but the ratio runs 1.06 to 9.64 and captures road indirectness.
- **Categoricals:** pickup, delivery, equipment. Native categoricals for LightGBM, one-hot for ridge. Raw coordinates are dropped, being a lookup of the city name.

- **Date:** day of week, day of year, month, cyclical sine/cosine pairs, and days_since_2025-01-01 as the linear trend term. The trend term is the only feature that can carry the model past the end of training.
- **Market:** market_index and quote_signal.
- **Cleaning flags:** six indicators, so an imputed value is never mistaken for a measured one.

6. Model choice

Three models on the same three folds. The baseline is the mean rate per mile of the training fold multiplied by distance - it uses no feature except distance, so anything that cannot beat it has not earned its place.

Model	RMSE ($)	MAE ($)	MAPE (%)	MAPE spread
Flat $/mi baseline	705.0	291.0	12.13	0.39
Ridge on log target	**627.5**	**119.1**	**5.16**	**0.69**
LightGBM	630.1	130.7	5.87	2.29

Mean across the three folds; MAPE spread is max minus min, which is the honest cost of month-to-month drift.

Ridge wins, and that is the finding. The reason is visible in the LightGBM feature importances: distance carries a gain of 128,690 and log(distance) 36,010 against 1,264 for the next feature. The target is close to linear in log(distance) with a slope of 0.871, which ridge fits exactly and trees approximate with step functions. Three checks confirmed it is not an artefact of under-training: 800 rounds was actively overfitting (mean MAE 164 at 800 rounds, 140 at 200, 124 at 100); early stopping inside each fold improved LightGBM but not enough; and linear_tree=True, which gives leaves linear models, was worse again at MAE 138.

A second reason favours the linear model here. A tree cannot extrapolate: every days_since_start value beyond the end of training lands in the same terminal leaf as the last training day, so the trend term flattens exactly where it is needed. Validation sits 61 days past that boundary. Ridge carries the trend forward in a straight line.

Per-fold detail

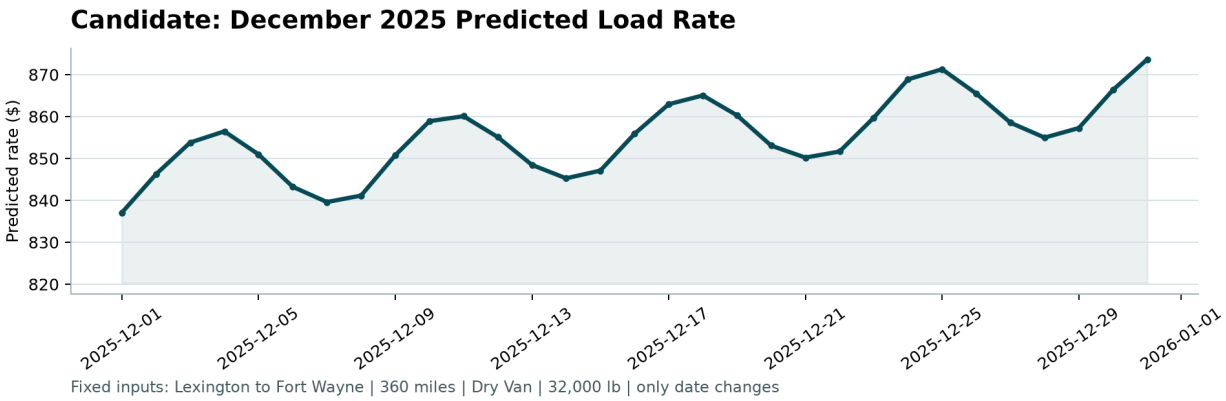
Fold	Ridge RMSE	Ridge MAE	Ridge MAPE	GBM MAE	GBM MAPE
fold 1 (2025-08)	614.0	108.4	4.89	156.2	7.15
fold 2 (2025-09)	622.6	123.8	5.01	115.6	4.86
fold 3 (2025-10)	645.9	125.1	5.58	120.2	5.60

7. Final fit and the 12,000 predictions

Ridge (alpha = 10, chosen by a fold sweep that moved MAPE by under 0.03 points, so the penalty is immaterial here) was refitted on all 48,000 training rows, trimmed as the folds were, and used to score validation.csv. Predicted rates run $118.08 to $7,040.67 with a mean of $2,432.88, against a training mean of $2,374 at a near-identical mean distance. Implied rate per mile is 1.83 / 2.21 / 3.06 $/mi at p1 / p50 / p99, inside the training band. No prediction hit the positive floor.

8. The December sensitivity curve

All 31 rows are the same load - Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb - with only the date changing, so the curve shows nothing but what the model learned about time. The input file carries six columns and the model needs more, and both gaps were closed without forecasting: coordinates come from the exact city lookup, and the market signals come from the 6,164 real December rows already present in validation.csv, taken as a per-date median. The chart therefore comes off the same model as the submission rather than a separate reduced-feature one.



candidate_december.png, produced by score.py.

Check	Value	Reading
Curve range	$837 - $874	Mean $855
Historical lane	$848	32 real loads at 2.355 $/mi x 360 mi
Gap to benchmark	+0.9%	Feature reconstruction is sound
Peak-to-trough swing	4.27%	Not flat: date features reached the model
Mean day-to-day step	0.63%	Not a sawtooth: date features are not overfit
Midweek - weekend	$+11.71	The day-of-week effect from section 1, intact
1 Dec to 31 Dec	$+36.55	Linear trend extrapolation

The shape is a weekly ripple on a slow trend, which is what section 1 predicted it should be.

9. What this model gets wrong

- **The trend is extrapolated, not observed.** The curve rises $37 across December purely because ridge continues the linear trend 61 days past the end of training. If the real market flattened or turned in Q4, this is confidently wrong in a way a tree's flat line would not have been. It is the single largest assumption in the submission.
- **RMSE barely separates the models** (705 to 628 to 630) because a few very long hauls dominate the squared term. MAE and MAPE are where the models actually differ, which is why all three are reported per fold rather than blended into one number.
- **1,447 validation rows involve a city never seen in training.** They are flagged and fall back on the remaining features, but they carry more uncertainty than the fold metrics suggest, since every fold was scored on cities the model had already seen.
- **These numbers come from held-out months, not from the scorer.** score.py validates format and renders the chart; it computes no error metric. Spotter calculates the final figures after submission.

10. Reproducing this

```
pip install -r requirements.txt
python predict.py # fits ridge, writes validation_predictions.csv
python chart.py # fills december-chart-inputs.csv
python score.py --predictions validation_predictions.csv \
--december-predictions december-chart-inputs.csv
```

Each module runs standalone and prints the evidence behind its own decisions: clean_data.py, build_features.py, target_transform.py, splits.py, models.py, predict.py, chart.py. make_report.py recomputes every figure in this document.